

ESPRIT — École Supérieure Privée d'Ingénierie et de
Technologies

Département : Technologies de l'Information

Rapport de Projet de Fin d'Études

Pour l'obtention du Diplôme National d'Ingénieur

Conception et Développement d'une Plateforme DevSecOps de Monitoring et de Gestion à Distance d'Infrastructure



Entreprise d'accueil : **CLEDISS**

Réalisé par :
Mariem Chaabani

Encadré par :
Encadrant académique : *[À compléter]*
Encadrant professionnel (CLEDISS) :
[À compléter]

Année Universitaire : **2025 – 2026**

Table des matières

Table des figures

Liste des tableaux

Introduction Générale

Dans un contexte où les infrastructures informatiques des entreprises reposent de plus en plus sur des architectures distribuées — serveurs on-premise et instances hébergées chez des fournisseurs cloud (IaaS/PaaS) — la capacité à superviser en temps réel l'état de santé de ces ressources et à intervenir à distance de manière sécurisée est devenue un enjeu opérationnel majeur. Une panne non détectée, une saturation de ressources non anticipée ou un service applicatif interrompu peuvent avoir des répercussions directes sur la continuité de service et, par conséquent, sur l'activité de l'entreprise.

C'est dans cette optique que s'inscrit le présent projet de fin d'études, réalisé au sein de l'entreprise **CLEDISS**, éditeur tunisien de solutions logicielles SaaS. L'objectif est de concevoir et de développer une plateforme complète de monitoring et de gestion à distance d'infrastructure serveur, accessible à la fois via une application web et une application mobile, consommant un socle applicatif commun (backend Node.js et base de données **MongoDB Atlas**), déployés sur un serveur **VPS OVH sous Debian Linux**. Au-delà du développement fonctionnel de la plateforme, le projet couvre également sa mise en production selon une démarche **DevSecOps** : conteneurisation, intégration continue avec analyse de qualité et de sécurité, déploiement orchestré sur Kubernetes, et supervision de l'infrastructure elle-même au moyen de Prometheus et Grafana.

Le projet a été mené selon une méthodologie agile **Scrum**, organisée en quatre sprints de développement, précédés d'un Sprint 0 d'analyse, chacun correspondant à un livrable fonctionnel ou technique cohérent. Cette organisation structure naturellement le présent rapport.

Le **premier chapitre** présente le cadre général du projet : l'organisme d'accueil CLEDISS et son produit phare Nomadis ERP, la problématique à l'origine du projet, une critique de l'existant, ainsi que la méthodologie de travail retenue.

Le **deuxième chapitre** correspond au Sprint 0, consacré à l'analyse et à la spécification des besoins : identification des acteurs, recueil des besoins fonctionnels et non fonctionnels, backlog du produit, architecture globale de la solution et choix technologiques.

Les **chapitres trois et quatre** détaillent les Sprints 1 et 2, dédiés à la construction de l'application de monitoring proprement dite : collecte de métriques par un agent Python, tableau de bord temps réel, authentification JWT, système d'alertes par email, actions à distance sécurisées via SSH, sauvegardes automatisées et application mobile React Native.

Enfin, les **chapitres cinq et six** présentent les Sprints 3 et 4, consacrés au volet DevSecOps du projet : conteneurisation avec Docker, chaîne d'intégration continue avec Jenkins (7 stages), analyse de qualité de code avec SonarQube et de vulnérabilités avec Trivy, puis déploiement et exploitation sur un cluster Kubernetes k3s avec GitOps (Argo CD), gestion des secrets (HashiCorp Vault), contrôle d'accès (RBAC) et supervision d'infrastructure (Prometheus + Grafana).

Le rapport se conclut par un bilan général du travail réalisé et les perspectives d'évolution envisageables pour la plateforme.

Cadre Général du Projet

Introduction

Ce premier chapitre a pour objectif de situer le projet dans son contexte. Nous présentons dans un premier temps l'organisme d'accueil, CLEDISS, ainsi que son produit phare Nomadis ERP. Nous exposons ensuite la problématique à l'origine du projet, une critique de l'existant, ainsi que la solution proposée. Enfin, nous détaillons la méthodologie de travail Scrum retenue pour la conduite du projet.

1.1 Présentation de l'organisme d'accueil

1.1.1 CLEDISS

CLEDISS est une entreprise tunisienne éditrice de logiciels, fondée en 2018 et spécialisée dans la conception de solutions SaaS pour la digitalisation des processus métier (force de vente, logistique, RH, facturation, inventaire). Positionnée sur le marché des progiciels de gestion intégrés, CLEDISS accompagne ses clients dans la transformation numérique de leurs activités en proposant une plateforme hébergée dans le cloud, accessible à distance et maintenue en continu par les équipes internes de l'entreprise.

TABLE 1.1 – Fiche d'identité de l'organisme d'accueil

Raison sociale	CLEDISS
Secteur d'activité	Édition de logiciels SaaS
Année de création	2018
Produit phare	Nomadis ERP
Localisation	Tunis, Tunisie
Site web	www.clediss.com

1.1.2 Produit phare : Nomadis ERP

Nomadis ERP constitue le produit central de l'offre de CLEDISS. Il s'agit d'un progiciel de gestion intégré (*Enterprise Resource Planning*) délivré en mode SaaS, permettant aux entreprises clientes de centraliser la gestion de leurs processus opérationnels

— gestion commerciale, suivi de stock, comptabilité, ressources humaines — au sein d’une plateforme unique accessible via le web.

Le mode de distribution SaaS implique que CLEDISS héberge et exploite elle-même l’infrastructure serveur sur laquelle s’exécute Nomadis ERP pour l’ensemble de ses clients. Cette responsabilité d’hébergement place naturellement la supervision de l’infrastructure — disponibilité des serveurs, performance, sauvegarde des données — au cœur des préoccupations opérationnelles de l’entreprise, et constitue le point de départ du présent projet.

1.1.3 Services proposés

En complément de l’édition logicielle, CLEDISS propose à ses clients un ensemble de services associés à l’exploitation de la plateforme Nomadis ERP :

- hébergement et exploitation de l’infrastructure serveur ;
- paramétrage et accompagnement au déploiement de la solution ;
- support technique et maintenance applicative ;
- évolutions fonctionnelles à la demande des clients.

1.2 Cadre général du projet

1.2.1 Contexte et problématique

L’hébergement d’une plateforme SaaS destinée à plusieurs entreprises clientes implique pour CLEDISS la gestion d’un parc de serveurs — physiques ou hébergés chez un fournisseur cloud — sur lesquels s’exécutent les différents composants applicatifs de Nomadis ERP. Or, à l’origine du projet, cette supervision reposait essentiellement sur des vérifications manuelles : connexion périodique aux serveurs pour contrôler leur charge, absence d’historisation des métriques, absence de mécanisme d’alerte automatique, et absence d’outil centralisé permettant d’intervenir à distance sur un service défaillant sans ouvrir une session SSH manuelle.

Cette approche pose plusieurs problèmes opérationnels :

- **détection tardive des incidents** — une saturation de ressources (CPU, mémoire, disque) n’est identifiée qu’a posteriori ;
- **absence de traçabilité** — aucune historisation des métriques ni des interventions effectuées sur les serveurs ;
- **réactivité limitée** — toute action corrective nécessite une connexion manuelle, ce qui allonge le délai de résolution ;

- **absence de vision consolidée** — aucun tableau de bord unique ne permet de visualiser l'état de l'ensemble du parc de serveurs.

La problématique centrale du projet est donc la suivante :

« Comment surveiller en temps réel l'état des serveurs, détecter les anomalies et intervenir à distance de manière sécurisée ? Et comment automatiser et sécuriser le déploiement de cette solution, du code jusqu'à la mise en production ? »

1.2.2 Étude de l'existant

Plusieurs outils de supervision existent sur le marché. Le ?? synthétise l'analyse des solutions les plus utilisées.

TABLE 1.2 – Étude comparative des solutions de supervision existantes

Outil	Points forts	Limites
Zabbix	Solution open source, monitoring complet.	Interface complexe, difficile à configurer pour des équipes non spécialisées.
Nagios	Outil très stable, système d'alertes par email.	Interface vieillissante, aucune application mobile.
Grafana + Prometheus	Combinaison moderne, dashboards visuels avancés.	Aucun module d'actions à distance, aucune application mobile, aucune gestion de backup.

1.2.3 Critique de l'existant

Aucune de ces solutions ne répond à l'ensemble des besoins de CLEDISS :

- aucune n'intègre d'actions à distance (redémarrage de services / serveurs) ;
- aucune ne propose une application mobile native ;
- aucune ne gère les sauvegardes automatiques avec suivi et historique ;
- aucune n'est intégrée dans une chaîne DevSecOps complète.

Ces constats mettent en évidence la nécessité de disposer d'un outil de supervision centralisé, temps réel et accessible depuis n'importe quel appareil.

1.2.4 Solution proposée

Pour répondre à ces constats, le projet propose la conception et le développement d'une plateforme de monitoring et de gestion à distance, structurée autour des principes

suivants :

- un **agent de collecte léger** développé en Python (`psutil`), installé sur chaque serveur à superviser, collectant les métriques toutes les 5 secondes (CPU, RAM, disque, réseau, uptime) ainsi que la liste des services actifs, transmises à un backend centralisé ;
- un **backend centralisé** Node.js/Express exposant une API REST unique, avec communication temps réel via WebSocket (Socket.io), consommée indifféremment par un client web et un client mobile ;
- un **système d'alertes** basé sur des seuils configurables (**WARNING** à 80 %, **CRITICAL** à 90 %), avec notification automatique par courrier électronique ;
- un module d'**actions à distance** sécurisé, permettant à un administrateur authentifié de redémarrer un service ou un serveur directement depuis l'interface, sans ouverture manuelle d'une session SSH ;
- un module de **sauvegarde automatisée**, planifiée quotidiennement à minuit, avec suivi et notification par email ;
- une industrialisation du déploiement selon une démarche **DevSecOps** : conteneurisation Docker, pipeline CI/CD Jenkins (7 stages), déploiement Kubernetes k3s, GitOps avec Argo CD, gestion des secrets avec HashiCorp Vault, contrôle d'accès RBAC, et supervision avec Prometheus et Grafana.

1.3 Méthodologie de travail

1.3.1 Choix du cadre méthodologique (Scrum)

Le projet a été conduit selon le cadre agile **Scrum**. Ce choix se justifie par la nature du projet, qui combine développement fonctionnel évolutif et mise en place progressive d'une chaîne d'exploitation technique : le découpage en sprints permet de livrer, à intervalles réguliers, des incréments fonctionnels testables et de réajuster les priorités du backlog en fonction des retours obtenus.

Le projet a été structuré en cinq phases : un Sprint 0 d'analyse et quatre sprints de développement :

- **Sprint 0** — analyse et spécification des besoins ;
- **Sprint 1** — application de monitoring, première partie (backend Node.js, agent Python de collecte, tableau de bord temps réel, authentification JWT) ;
- **Sprint 2** — application de monitoring, seconde partie (alertes email, actions à distance SSH, sauvegardes automatisées, application mobile React Native) ;

- **Sprint 3** — architecture DevSecOps, première partie (conteneurisation Docker, pipeline CI/CD Jenkins 7 stages, analyse qualité SonarQube, scan vulnérabilités Trivy);
- **Sprint 4** — architecture DevSecOps, seconde partie (orchestration Kubernetes k3s, GitOps Argo CD, gestion des secrets HashiCorp Vault, RBAC, supervision Prometheus + Grafana).

1.3.2 Les rôles dans Scrum

Le cadre Scrum définit trois rôles principaux, adaptés dans le contexte de ce projet de fin d'études :

TABLE 1.3 – Rôles Scrum appliqués au projet

Rôle	Responsabilité dans le cadre du projet
Product Owner	Encadrant professionnel CLEDISS — définit et priorise les besoins métier, valide les incréments livrés à l'issue de chaque sprint.
Scrum Master / Développeuse	Mariem Chaabani, étudiante en charge du projet — anime le déroulement des sprints, développe les incréments, assure le suivi de l'avancement.
Équipe de développement	Mariem Chaabani, avec le support technique de l'encadrant académique et de l'encadrant professionnel.

1.3.3 Planning du projet (Diagramme de Gantt)

Le planning prévisionnel du projet, structuré autour des cinq phases définies précédemment et s'étendant de mars à août 2026, est présenté à la ??.

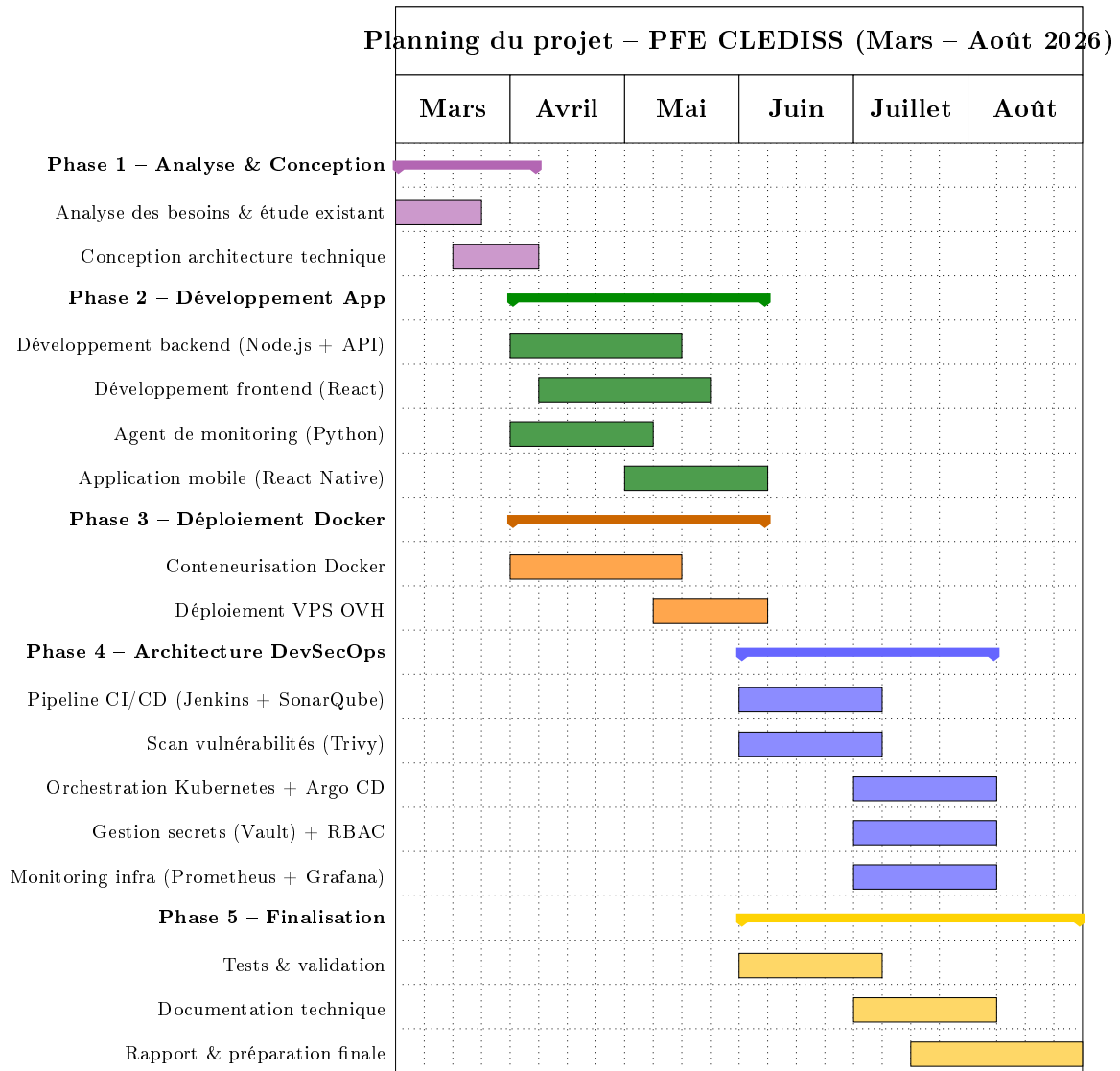


FIGURE 1.1 – Diagramme de Gantt prévisionnel du projet

Conclusion

Ce chapitre a permis de situer le projet dans son contexte organisationnel et de poser la problématique à laquelle il répond, ainsi que la démarche méthodologique retenue pour y répondre. Le chapitre suivant détaille le Sprint 0, consacré à l'analyse et à la spécification des besoins, posant ainsi les fondations de la solution développée.

Sprint 0 – Analyse et Spécification des Besoins

Introduction

Ce chapitre présente les travaux du Sprint 0, consacré à l'analyse et à la spécification des besoins du projet. Nous y identifions les acteurs du système, recensons les besoins fonctionnels et non fonctionnels, présentons le backlog du produit ainsi que les diagrammes de conception préliminaires (cas d'utilisation global, classes), avant de détailler l'environnement de travail et l'architecture globale retenue.

2.1 Identification des acteurs

Le système met en jeu deux acteurs principaux, l'un humain, l'autre automatisé :

- **Administrateur** — utilisateur authentifié de la plateforme (via l'application web ou mobile), responsable de la supervision des serveurs, de la gestion des alertes, et de l'exécution des actions de maintenance à distance ;
- **Agent de supervision** — programme Python exécuté de manière autonome sur chaque serveur supervisé, qui collecte périodiquement les métriques système toutes les 5 secondes et les transmet au backend via API REST, en continuant de fonctionner même si le backend est temporairement indisponible.

2.2 Étude des besoins

2.2.1 Besoins fonctionnels

L'étude des besoins a permis d'identifier les fonctionnalités suivantes, accessibles depuis l'application web et mobile :

BF1 — Authentification. Connexion sécurisée de l'administrateur par email et mot de passe, avec émission d'un jeton d'accès JWT stocké côté client et validé à chaque requête sensible.

BF2 — Tableau de bord global. Visualisation consolidée de l'ensemble des serveurs supervisés, avec leur statut en temps réel (**OK**, **WARNING**, **CRITICAL**, **OFFLINE**), un résumé des métriques globales et le nombre d'alertes actives.

BF3 — Détail d'un serveur. Consultation des métriques en temps réel (CPU, RAM, disque, réseau, uptime) et de leur historique sous forme de courbes graphiques pour un serveur donné. Rafraîchissement automatique toutes les 5 secondes.

BF4 — Gestion des alertes. Génération automatique d'alertes lors du dépassement de seuils configurables (**WARNING** à 80 %, **CRITICAL** à 90 % pour CPU, RAM et disque), notification par courrier électronique, consultation et acquittement des alertes actives depuis l'interface.

BF5 — Détection automatique des services. Identification, par l'agent Python, des services `systemd` actifs sur chaque serveur supervisé (`nginx`, `apache2`, `mongod`, `pm2`, ...), sans configuration manuelle préalable, et transmission de la liste au backend à chaque envoi de métriques.

BF6 — Actions à distance. Redémarrage ou arrêt d'un service applicatif, redémarrage complet d'un serveur, exécutés via une connexion SSH sécurisée initiée par le backend avec les identifiants stockés en base de données.

BF7 — Journalisation et audit. Traçabilité complète de toute action d'administration exécutée à distance : auteur, cible, commande exécutée, horodatage et résultat. Accessible via un journal d'audit dans l'interface.

BF8 — Sauvegardes automatisées. Exécution planifiée quotidienne de sauvegardes à minuit, avec historique consultable et notification du résultat (succès ou échec) par courrier électronique.

BF9 — Notifications temps réel. Réception de notifications en temps réel sur l'application en cas d'alerte critique, via WebSocket (Socket.io).

BF10 — Application mobile. Accès à l'ensemble des fonctionnalités (tableau de bord, alertes, actions à distance) depuis une application mobile React Native, consommant le même backend que l'application web.

2.2.2 Besoins non fonctionnels

BNF1 — Performance. Les métriques sont collectées toutes les 5 secondes par l'agent Python et affichées en temps réel dans le tableau de bord via WebSocket. Le délai de rafraîchissement de l'interface est inférieur à 1 seconde.

BNF2 — Sécurité. Les actions sensibles (actions à distance, acquittement d'alertes) sont protégées par une double vérification : validité du jeton JWT et appartenance de l'utilisateur au rôle administrateur, vérifiée en base de données. Toutes les actions sont journalisées dans un journal d'audit.

BNF3 — Disponibilité. L'ensemble des composants (frontend, backend, base de données) est déployé sur un cluster Kubernetes k3s avec redémarrage automatique des conteneurs en cas de défaillance, garantissant une disponibilité continue.

BNF4 — Scalabilité. L'architecture multi-serveurs permet l'ajout de nouveaux serveurs supervisés sans modification du backend : il suffit d'installer et de configurer l'agent Python sur le nouveau serveur pour qu'il apparaisse automatiquement dans le tableau de bord.

BNF5 — Portabilité. L'ensemble des fonctionnalités est accessible de façon équivalente depuis un client web (React.js) et un client mobile (React Native), consommant la même API REST centralisée.

BNF6 — Maintenabilité. Le backend est structuré de façon modulaire (routes, modèles, services, middlewares), afin de faciliter l'évolution et la maintenance du code. La qualité du code est contrôlée automatiquement par SonarQube à chaque déploiement.

BNF7 — Ergonomie. L'interface est entièrement en français, cohérente visuellement entre le web et le mobile, avec un thème sombre professionnel aux couleurs de CLEDISS.

BNF8 — Industrialisation du déploiement. La mise en production est reproductible et automatisée via un pipeline CI/CD Jenkins (7 stages), incluant analyse de qualité (SonarQube), scan de vulnérabilités (Trivy), conteneurisation (Docker), déploiement (Kubernetes k3s) et GitOps (Argo CD).

BNF9 — Résilience de l’agent. L’agent Python continue de collecter les métriques localement même si le backend est temporairement indisponible, et reprend l’envoi dès que la connexion est rétablie.

2.3 Diagramme de cas d’utilisation global

La ?? présente le diagramme de cas d’utilisation global de la plateforme, organisé par domaine fonctionnel : authentification (gris), supervision (bleu), alertes (orange), actions à distance (violet), sauvegardes et notifications (vert).

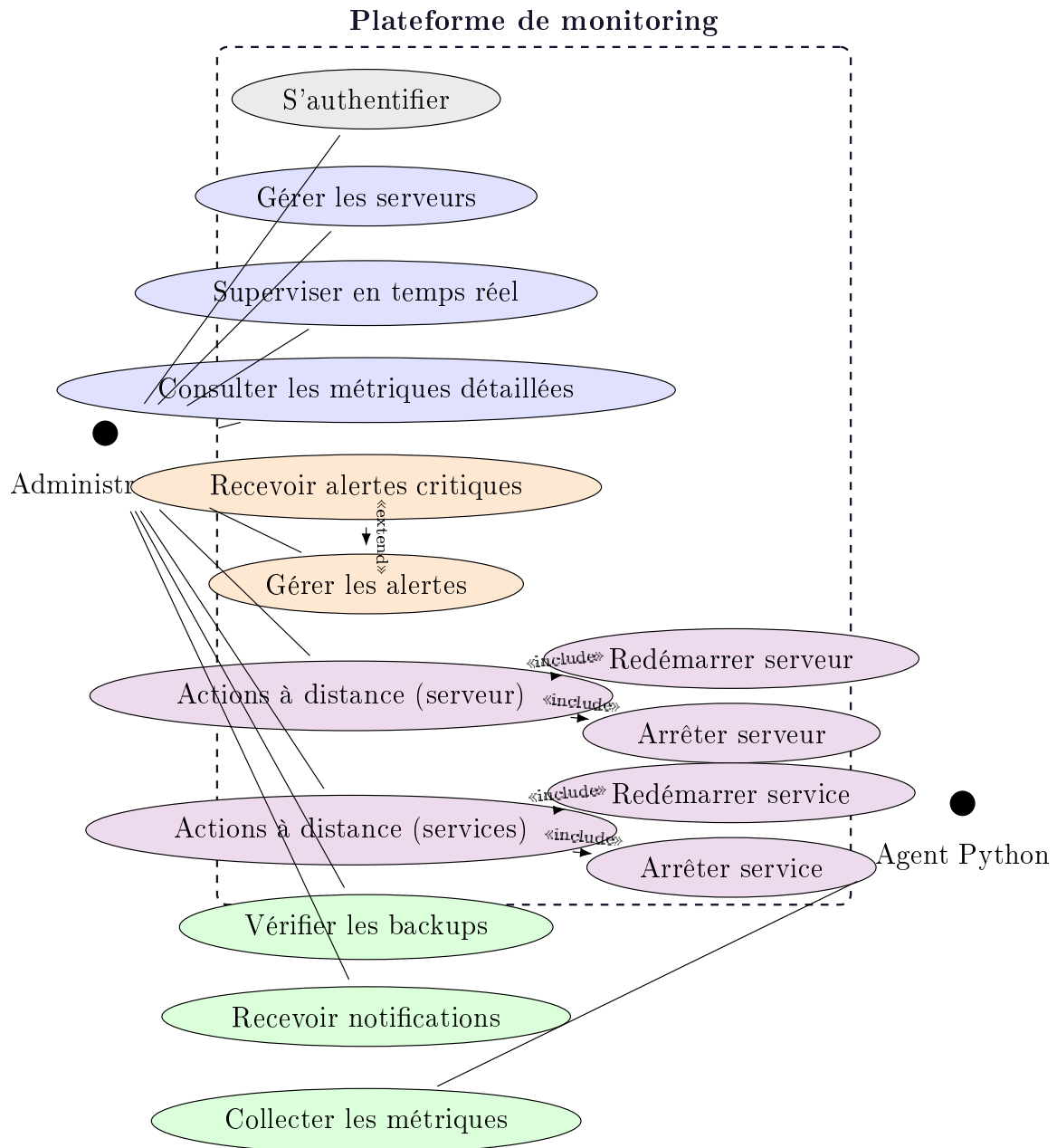


FIGURE 2.1 – Diagramme de cas d’utilisation global

2.4 Backlog du produit

Le ?? présente un extrait représentatif du backlog du produit, exprimé sous forme de user stories.

TABLE 2.1 – Extrait du backlog du produit

ID	User Story	Priorité
US01	En tant qu’administrateur, je veux me connecter avec mon email et mon mot de passe afin d’accéder à la plateforme de manière sécurisée.	Élevée
US02	En tant qu’administrateur, je veux visualiser la liste des serveurs et leur statut (OK , WARNING , CRITICAL , OFFLINE) afin d’identifier rapidement un problème.	Élevée
US03	En tant qu’administrateur, je veux consulter l’historique des métriques d’un serveur (CPU, RAM, disque, réseau) sous forme de graphique afin d’analyser son évolution.	Élevée
US04	En tant qu’administrateur, je veux recevoir un email automatique lorsqu’un seuil critique est dépassé (WARNING 80 %, CRITICAL 90 %) afin d’être alerté sans consulter le tableau de bord.	Élevée
US05	En tant qu’administrateur, je veux acquitter une alerte afin d’indiquer qu’elle a été prise en charge.	Moyenne
US06	En tant qu’administrateur, je veux redémarrer ou arrêter un service à distance afin de résoudre un incident sans ouvrir de session SSH manuelle.	Élevée
US07	En tant qu’administrateur, je veux que les services affichés correspondent aux services réellement actifs sur le serveur afin d’éviter toute action sur un service inexistant.	Moyenne
US08	En tant qu’administrateur, je veux que les sauvegardes s’exécutent automatiquement chaque nuit à minuit et recevoir un email de confirmation afin de ne pas dépendre d’une action manuelle.	Moyenne
US09	En tant qu’administrateur, je veux accéder aux mêmes fonctionnalités depuis mon téléphone afin d’intervenir même hors de mon poste de travail.	Moyenne
US10	En tant qu’administrateur, je veux redémarrer complètement un serveur à distance afin de résoudre une panne critique sans intervention physique.	Élevée
US11	En tant qu’administrateur, je veux consulter le journal d’audit des actions effectuées afin d’assurer la traçabilité des interventions.	Moyenne
US12	En tant que responsable technique, je veux que chaque déploiement soit précédé d’une analyse de qualité (SonarQube) et de sécurité (Trivy) afin de limiter les régressions en production.	Moyenne

Table 2.1 – (suite)

ID	User Story	Priorité
US13	En tant que responsable technique, je veux que le déploiement soit automatisé via un pipeline CI/CD Jenkins afin de réduire les erreurs manuelles de mise en production.	Moyenne
US14	En tant que responsable technique, je veux superviser l'infrastructure Kubernetes (pods, ressources) via Prometheus et Grafana afin de garantir la disponibilité de la plateforme.	Faible
US15	En tant que responsable technique, je veux que les secrets (mots de passe, tokens) soient gérés de manière sécurisée via HashiCorp Vault afin d'éviter toute exposition dans le code.	Faible

2.5 Diagramme de classes

Le ?? présente le diagramme de classes du domaine, reflétant les principales entités persistées (collections MongoDB, modélisées via Mongoose).

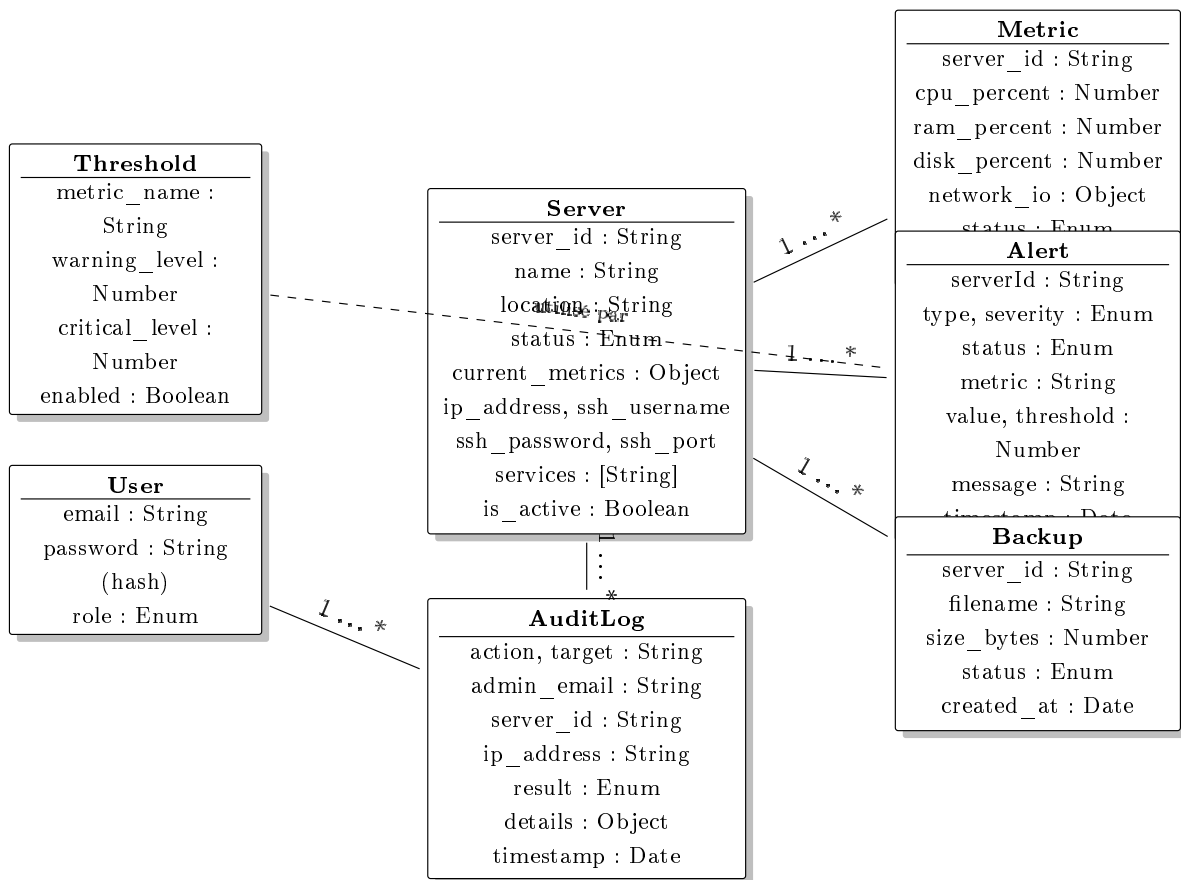


FIGURE 2.2 – Diagramme de classes du domaine

2.6 Environnement de travail

2.6.1 Langages et Frameworks utilisés

Node.js. Node.js est un environnement d'exécution JavaScript open source, multiplateforme, orienté événements et non bloquant. Il est particulièrement adapté au développement d'applications réseau temps réel à haute performance. Dans ce projet, Node.js constitue le socle du backend de la plateforme de monitoring [?].

Logo manquant
images/logo-nodejs.png

Express.js. Express.js est un framework web minimaliste et flexible pour Node.js, fournissant un ensemble de fonctionnalités robustes pour le développement d'API REST. Il est utilisé dans ce projet pour structurer les routes du backend et gérer les requêtes HTTP entrantes [?].

Logo manquant
images/logo-express.png

Socket.io. Socket.io est une bibliothèque JavaScript permettant la communication bidirectionnelle en temps réel entre un serveur Node.js et ses clients, via le protocole WebSocket. Elle est utilisée dans ce projet pour diffuser les métriques des serveurs en temps réel vers le tableau de bord [?].

Logo manquant
images/logo-socketio.png

React.js. React.js est une bibliothèque JavaScript open source, développée par Meta, dédiée à la construction d'interfaces utilisateur dynamiques et réactives. Elle repose sur un système de composants réutilisables et une gestion efficace du DOM virtuel. Dans ce projet, React.js est utilisé pour développer le frontend web de la plateforme de monitoring [?].

Logo manquant
images/logo-react.png

React Native. React Native est un framework JavaScript open source, développé par Meta, permettant de créer des applications mobiles natives pour iOS et Android à partir d'une base de code unique. Dans ce projet, React Native est utilisé conjointement avec Expo pour développer l'application mobile de monitoring [?].

Logo manquant
images/logo-reactnative.png

Expo. Expo est une plateforme open source construite autour de React Native, qui simplifie le développement, le test et le déploiement d'applications mobiles. Elle fournit un ensemble d'outils et de bibliothèques prêts à l'emploi. Dans ce projet, Expo est utilisé pour faciliter le développement et le test de l'application mobile [?].

Logo manquant
images/logo-expo.png

Python. Python est un langage de programmation interprété, multiparadigme et multiplateforme, reconnu pour sa lisibilité et sa richesse en bibliothèques. Dans ce projet, Python est utilisé pour développer l'agent de supervision installé sur chaque serveur supervisé [?].

Logo manquant
images/logo-python.png

MongoDB Atlas. MongoDB Atlas est un service de base de données cloud entièrement géré, basé sur MongoDB, un système de gestion de bases de données NoSQL orienté documents. Dans ce projet, MongoDB Atlas est utilisé comme base de données centralisée, hébergeant les métriques, alertes, sauvegardes et données utilisateurs de la plateforme [?].

Logo manquant
images/logo-mongodb-atlas.png

JWT (JSON Web Token). JSON Web Token est un standard ouvert (RFC 7519) définissant un format compact et autonome pour la transmission sécurisée d'informations entre parties sous forme de jeton signé numériquement. Dans ce projet, JWT est utilisé pour sécuriser l'authentification des administrateurs et protéger les routes sensibles de l'API [?].

Logo manquant
images/logo-jwt.png

2.6.2 Environnement logiciel

Visual Studio Code. Visual Studio Code, souvent appelé VS Code, est un éditeur de code source léger, gratuit et open source, développé par Microsoft. Il offre une grande richesse d'extensions et un support natif de nombreux langages de programmation. Dans ce projet, VS Code est l'environnement de développement principal utilisé pour coder l'ensemble des composants de la plateforme (backend, frontend, agent Python, application mobile).

Logo manquant
images/logo-vscode.png

Postman. Postman est une application permettant de tester, documenter et déboguer des API REST. Elle offre une interface graphique intuitive pour envoyer des requêtes HTTP et analyser les réponses. Dans ce projet, Postman est utilisé pour tester les endpoints de l'API backend durant la phase de développement.

Logo manquant
images/logo-postman.png

Expo Go. Expo Go est une application mobile disponible sur iOS et Android, permettant de tester en temps réel une application React Native développée avec Expo, sans avoir à compiler un fichier APK ou IPA. Dans ce projet, Expo Go est utilisé pour tester l'application mobile de monitoring directement sur un appareil physique durant la phase de développement.

Logo manquant
images/logo-expo-go.png

GitHub. GitHub est une plateforme d'hébergement de code source basée sur Git, permettant la gestion de versions, la collaboration et le partage de code. Dans ce projet, GitHub héberge le code source de la plateforme et constitue la source de vérité pour le déploiement GitOps via Argo CD.

Logo manquant
images/logo-github.png

MongoDB Compass. MongoDB Compass est une interface graphique officielle pour MongoDB, permettant de visualiser, explorer et manipuler les données stockées dans une base MongoDB. Dans ce projet, MongoDB Compass est utilisé pour inspecter et gérer les données de MongoDB Atlas durant la phase de développement et de tests.

Logo manquant
images/logo-mongodb-compass.png

Draw.io. Draw.io est un outil de conception de diagrammes en ligne, gratuit et open source, permettant de créer des diagrammes UML, des schémas d'architecture et des organigrammes. Dans ce projet, Draw.io est utilisé pour concevoir les diagrammes de cas d'utilisation, de séquence et d'architecture de la plateforme.

Logo manquant
images/logo-drawio.png

2.6.3 Environnement matériel

- poste de développement local ;
- serveur VPS OVH (Debian), hébergeant le cluster Kubernetes de production, adresse IP 141.227.129.194 ;
- machine virtuelle VirtualBox, utilisée comme second serveur supervisé pour valider le fonctionnement multi-serveurs ;
- smartphone / émulateur, pour les tests de l'application mobile.

2.7 Architecture globale

2.7.1 Architecture physique

La ?? présente l'architecture physique de la solution telle que déployée en production.

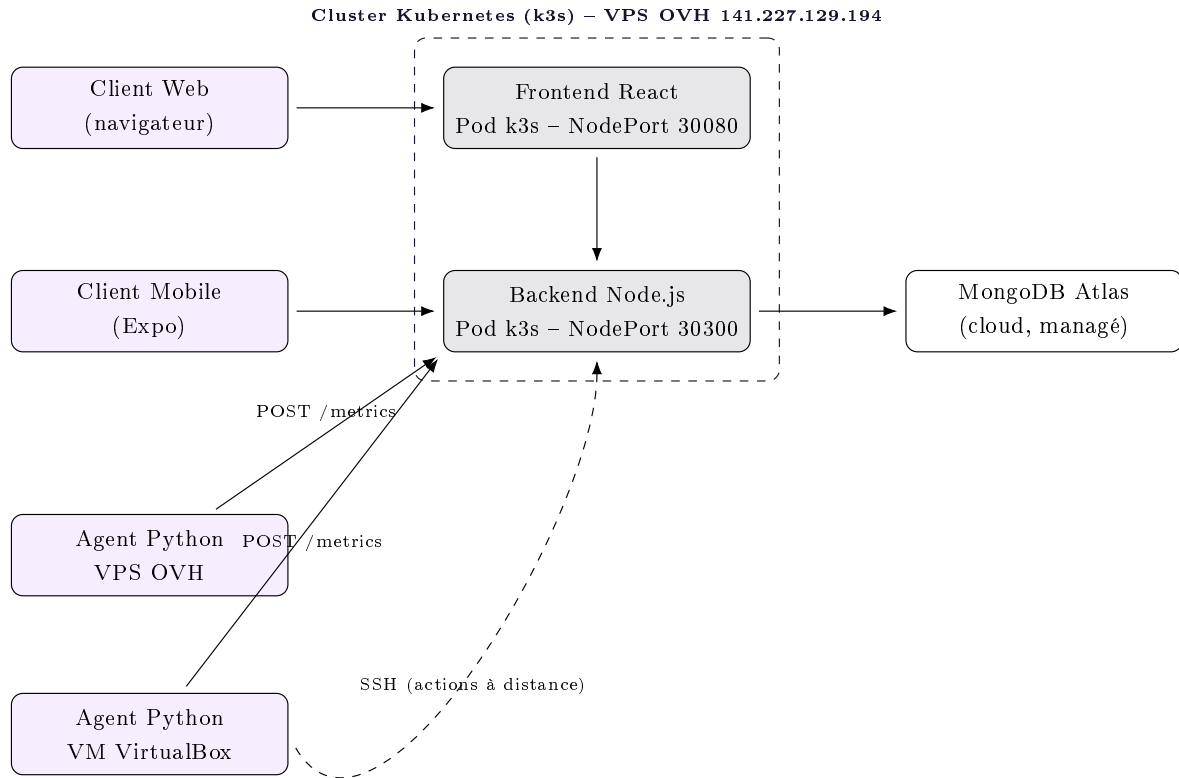


FIGURE 2.3 – Architecture physique de la plateforme

2.7.2 Architecture logique

D'un point de vue logique, la plateforme s'organise en trois couches, complétées par un canal de communication temps réel et un canal d'administration à distance :

- **couche présentation** — application web React et application mobile React Native, toutes deux consommant la même API REST ;
- **couche applicative** — API REST Express (authentification, serveurs, métriques, alertes, actions à distance, sauvegardes), complétée par un canal WebSocket pour la diffusion des mises à jour en temps réel ;
- **couche données** — base de données MongoDB (Atlas), organisée en collections correspondant aux entités du ?? ;
- **couche collecte** — agents Python déployés sur chaque serveur supervisé, transmettant leurs métriques à la couche applicative par requêtes HTTP périodiques ;
- **couche administration** — connexions SSH établies à la demande par le backend vers les serveurs supervisés, pour l'exécution des actions à distance.

2.8 Backlog général des sprints

Le ?? synthétise la répartition des objectifs entre les cinq sprints du projet.

TABLE 2.2 – Backlog général des sprints

Sprint	Objectifs principaux	Chapitre
Sprint 0	Analyse des besoins, architecture globale, choix technologiques	Chapitre 2
Sprint 1	Backend, agent de collecte, tableau de bord temps réel, authentification	Chapitre 3
Sprint 2	Alertes, actions à distance SSH, sauvegardes, application mobile	Chapitre 4
Sprint 3	Conteneurisation, intégration continue, qualité et sécurité du code	Chapitre 5
Sprint 4	Orchestration Kubernetes, GitOps, gestion des secrets, supervision d'infrastructure	Chapitre 6

Conclusion

Ce chapitre a permis de poser les fondations du projet : identification des acteurs, expression des besoins, architecture globale et environnement technique. Les chapitres suivants détaillent la réalisation de chacun des sprints identifiés dans le backlog général.

Sprint 1 – Application de Monitoring (Partie 1)

Introduction

Ce chapitre présente la réalisation du Sprint 1, consacré à la mise en place du socle de l'application de monitoring : architecture backend, agent Python de collecte des métriques, tableau de bord temps réel et authentification.

3.1 Objectif du sprint

L'objectif du Sprint 1 est de disposer, à son issue, d'une chaîne complète et fonctionnelle allant de la collecte d'une métrique système sur un serveur jusqu'à son affichage en temps réel dans un tableau de bord, l'ensemble étant protégé par une authentification.

3.2 Analyse du sprint 1

3.2.1 Sprint Backlog

TABLE 3.1 – Sprint Backlog – Sprint 1

ID	Tâche	Statut
T1.1	Modélisation des entités Server et Metric (Mongoose)	Terminé
T1.2	Mise en place de l'API REST (Express) et connexion à MongoDB Atlas	Terminé
T1.3	Développement de l'agent Python de collecte (psutil)	Terminé
T1.4	Endpoint POST /metrics de réception des métriques	Terminé
T1.5	Diffusion temps réel des mises à jour (WebSocket)	Terminé
T1.6	Développement du tableau de bord React (liste des serveurs, statut)	Terminé
T1.7	Mise en place de l'authentification JWT	Terminé

3.2.2 Diagramme de cas d'utilisation

Le ?? détaille les cas d'utilisation couverts par ce sprint.

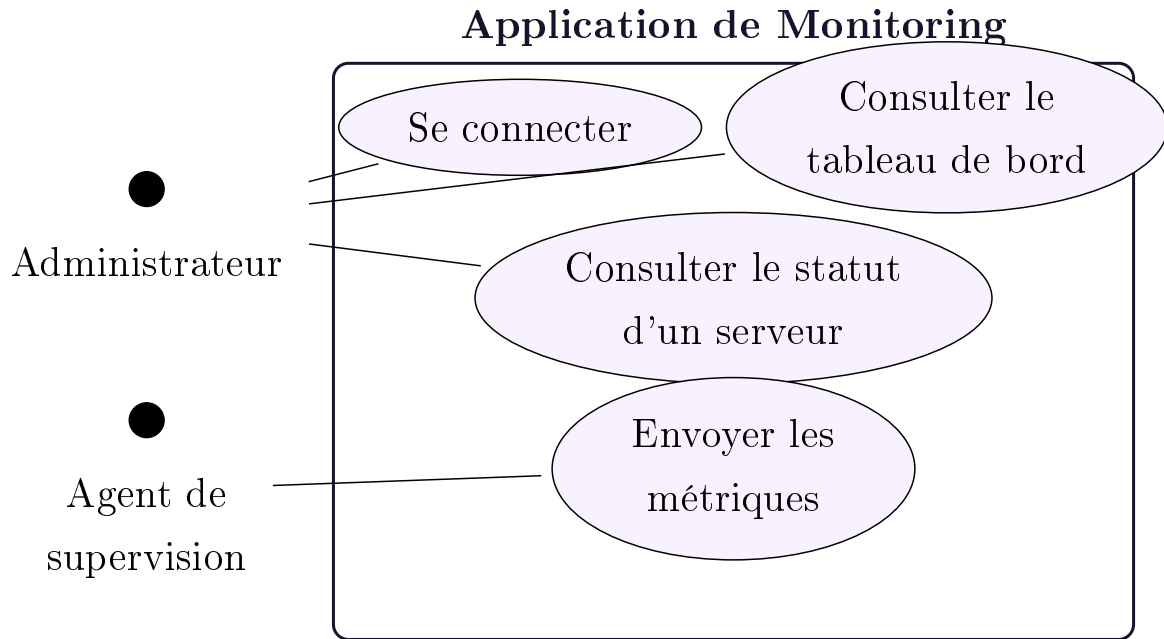


FIGURE 3.1 – Diagramme de cas d'utilisation – Sprint 1

3.2.3 Diagramme de séquence

Le ?? illustre le déroulement d'un cycle de collecte et de diffusion d'une métrique, du serveur supervisé jusqu'au tableau de bord.

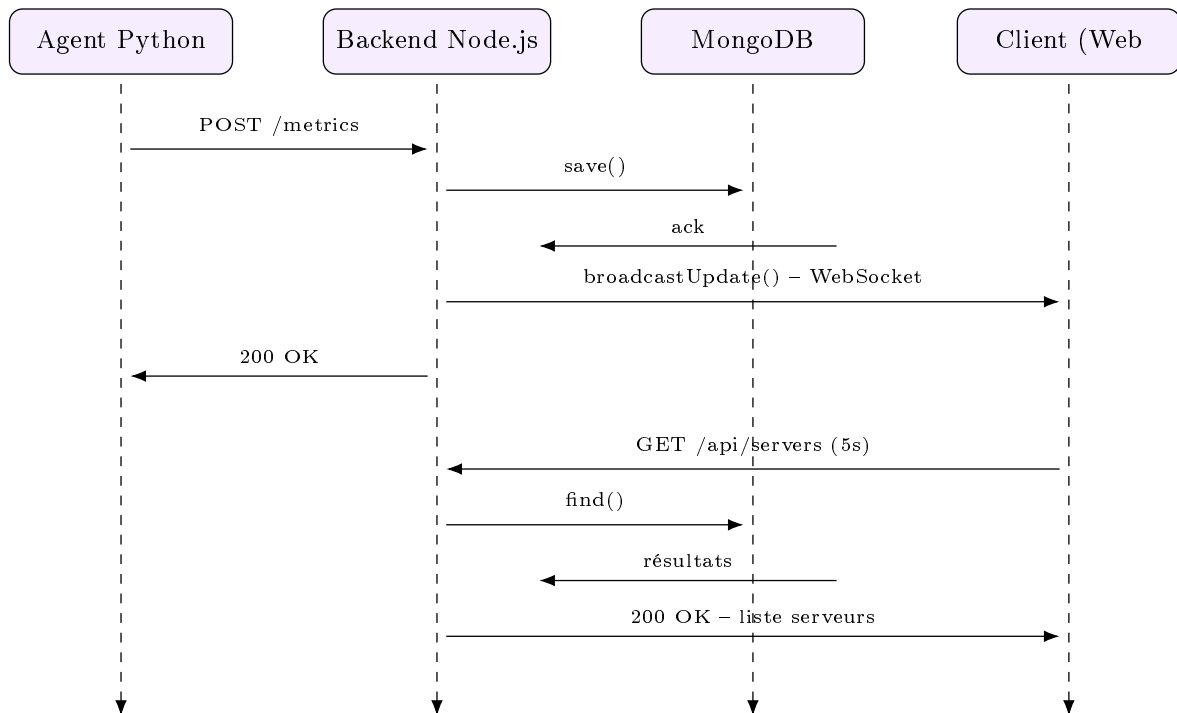


FIGURE 3.2 – Diagramme de séquence – Collecte et diffusion d'une métrique

3.3 Réalisation

3.3.1 Architecture backend (Node.js + Express)

Le backend est développé avec **Node.js** et le framework **Express**. Il est structuré de façon modulaire, selon une organisation en trois couches :

- **models/** — schémas Mongoose définissant la structure des documents MongoDB (**Server**, **Metric**, **Alert**, **User**, ...);
- **routes/** — définition des points d'accès de l'API REST, regroupés par domaine fonctionnel (**servers.js**, **metrics.js**, **alerts.js**, **remoteActions.js**, ...);
- **services/** — logique métier réutilisable (calcul de statut, gestion des alertes, envoi d'emails, ...), découplée des routes.

La persistance des données repose sur **MongoDB Atlas**, une offre managée dans le cloud, ce qui dispense l'équipe de l'administration d'une instance MongoDB auto-hébergée.

Le point d'entrée central de la collecte de métriques est l'endpoint **POST /metrics** : à chaque réception, le backend recherche (ou crée s'il n'existe pas encore) le document **Server** correspondant, calcule le statut du serveur à partir des seuils configurés (**StatusService**), persiste la métrique, puis diffuse la mise à jour aux clients connectés.

3.3.2 Agent Python de collecte des métriques

L'agent de supervision est un script **Python** exécuté en continu sur chaque serveur à superviser. Il s'appuie sur la bibliothèque **psutil** pour collecter, à intervalle régulier, les métriques système : taux d'utilisation du CPU, de la RAM, du disque, ainsi que des statistiques réseau et le temps de fonctionnement (*uptime*) du système.

À chaque cycle de collecte, l'agent transmet ces métriques au backend par une requête **HTTP POST** vers l'endpoint **/metrics**, accompagnées de l'identifiant et du nom du serveur (définis dans un fichier de configuration local). L'envoi s'appuie sur une session **HTTP** persistante, avec une logique de nouvelle tentative (*retry*) en cas d'échec temporaire de connexion au backend, afin de garantir la résilience de la collecte face aux interruptions réseau ponctuelles.

3.3.3 Dashboard temps réel

Le tableau de bord, développé en **React.js**, présente une vue consolidée de l'ensemble des serveurs supervisés : nom, statut (identifié par un code couleur), et métriques courantes. La mise à jour du tableau de bord s'appuie sur un canal **WebSocket** maintenu ouvert entre le client et le backend : dès qu'une nouvelle métrique est reçue et

traitée côté serveur, une notification est diffusée à l'ensemble des clients connectés, qui mettent alors à jour leur affichage sans nécessiter de rechargement de page.

3.3.4 Authentification JWT

L'accès à la plateforme est protégé par une authentification basée sur les **JSON Web Tokens (JWT)**. Le processus se déroule comme suit :

1. l'utilisateur soumet son email et son mot de passe à l'endpoint `POST /api/auth/login` ;
2. le backend vérifie les identifiants (le mot de passe étant stocké sous forme hachée) et, en cas de succès, génère un jeton JWT signé, contenant l'identifiant, l'email et le rôle de l'utilisateur, avec une durée de validité de 24 heures ;
3. ce jeton est ensuite transmis par le client dans l'en-tête `Authorization: Bearer <token>` de chaque requête adressée aux routes protégées ;
4. un middleware `verifyToken` intercepte ces requêtes, vérifie la validité et la signature du jeton, et rejette la requête (code HTTP 401) en cas de jeton absent, invalide ou expiré.

Ce mécanisme est commun aux clients web et mobile, garantissant un comportement d'authentification homogène quel que soit le point d'accès à la plateforme.

Conclusion

À l'issue de ce sprint, la chaîne complète de collecte et de visualisation des métriques est opérationnelle : un serveur supervisé transmet ses métriques via l'agent Python, celles-ci sont persistées et diffusées en temps réel par le backend, et affichées dans un tableau de bord web protégé par authentification. Le sprint suivant enrichit cette base avec le système d'alertes, les actions à distance, les sauvegardes et l'application mobile.

Sprint 2 – Application de Monitoring (Partie 2)

Introduction

Ce chapitre présente la réalisation du Sprint 2, qui complète l'application de monitoring construite au sprint précédent par les fonctionnalités d'administration : système d'alertes, actions à distance sécurisées, gestion des sauvegardes automatisées, ainsi que l'application mobile.

4.1 Objectif du sprint

L'objectif de ce sprint est de transformer la plateforme d'un outil de *visualisation* en un véritable outil d'*administration* : détecter automatiquement les situations anormales, permettre à l'administrateur d'intervenir directement depuis l'interface, garantir la disponibilité des données par des sauvegardes planifiées, et rendre l'ensemble de ces fonctionnalités accessibles depuis une application mobile.

4.2 Analyse du sprint 2

4.2.1 Sprint Backlog

TABLE 4.1 – Sprint Backlog – Sprint 2

ID	Tâche	Statut
T2.1	Calcul du statut serveur à partir de seuils configurables (StatusService)	Terminé
T2.2	Génération automatique d'alertes et notification par email (AlertService)	Terminé
T2.3	Détection automatique des services actifs par l'agent (systemctl)	Terminé
T2.4	Endpoints d'actions à distance sécurisées (redémarrage / arrêt de service, redémarrage serveur) via SSH	Terminé
T2.5	Journal d'audit des actions à distance	Terminé
T2.6	Planification et exécution automatisée des sauvegardes (node-cron)	Terminé
T2.7	Notification email du résultat des sauvegardes	Terminé
T2.8	Refonte complète de l'application mobile (React Native / Expo)	Terminé

4.2.2 Diagramme de cas d'utilisation

Le ?? présente les cas d'utilisation couverts par ce sprint.

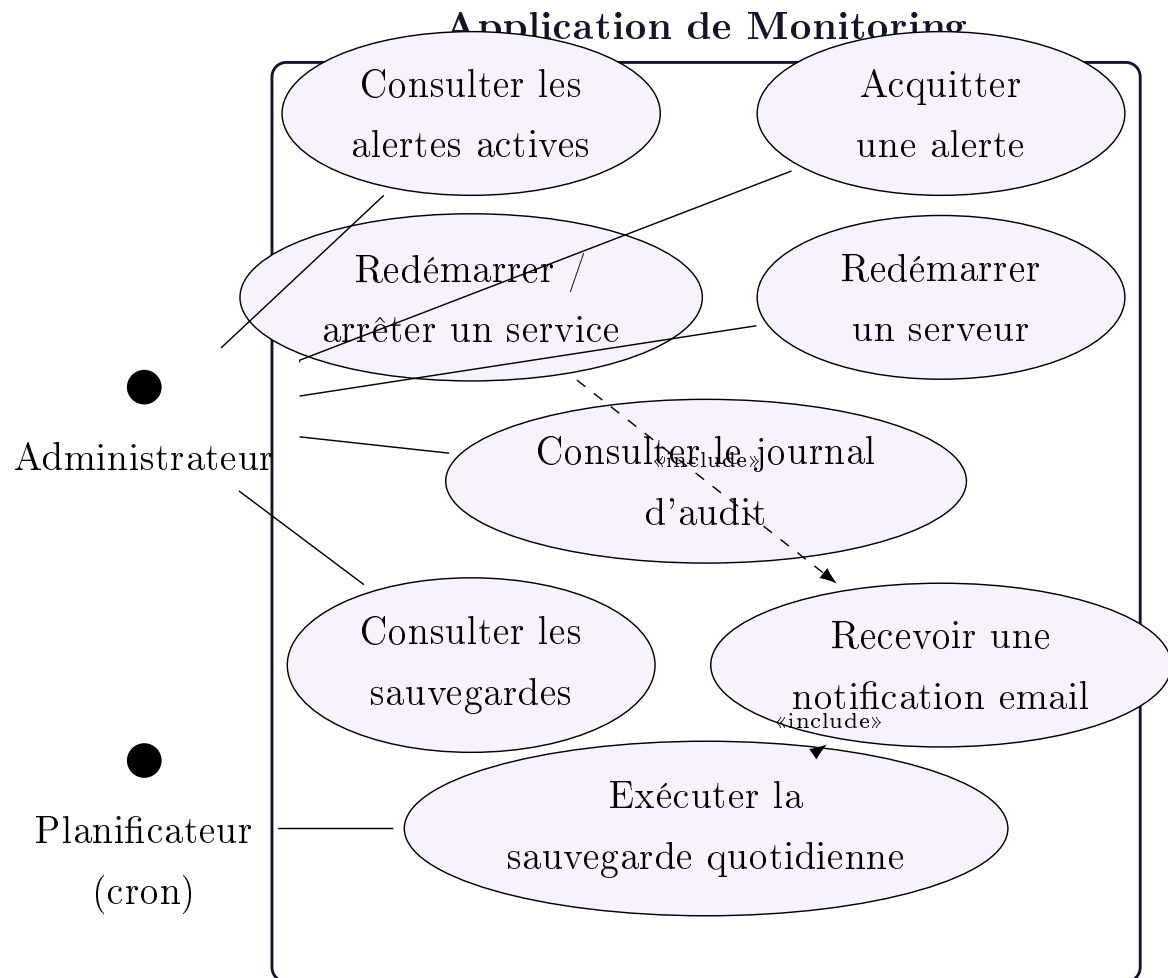


FIGURE 4.1 – Diagramme de cas d'utilisation – Sprint 2

4.2.3 Diagramme de séquence

Le ?? détaille le déroulement d'une action de redémarrage de service à distance, depuis le clic de l'administrateur jusqu'à la confirmation du résultat.

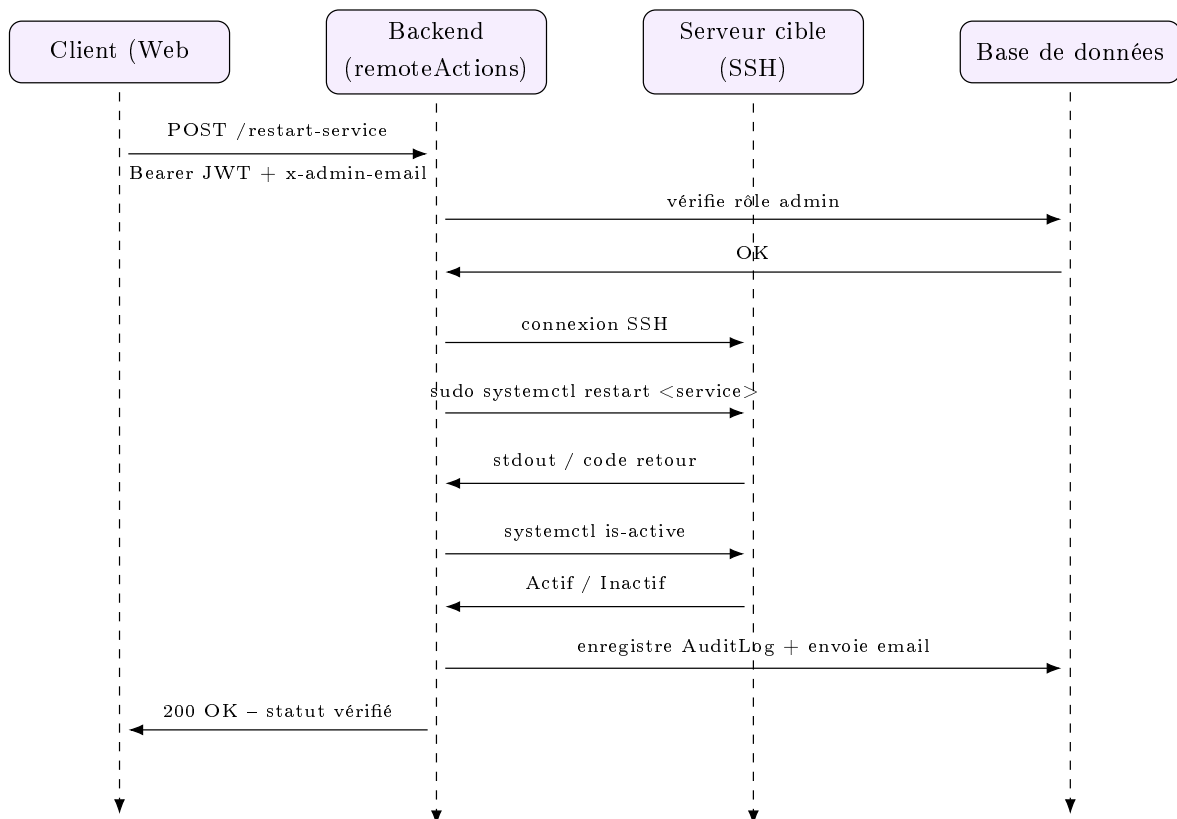


FIGURE 4.2 – Diagramme de séquence – Redémarrage d'un service à distance

4.3 Réalisation

4.3.1 Système d'alertes (CPU, RAM, Disque)

Le statut d'un serveur (OK, WARNING, CRITICAL) est calculé à chaque réception de métrique par le service `StatusService`, à partir de seuils configurables persistés en base de données (avec des valeurs par défaut de **80 %** pour le seuil d'alerte et **90 %** pour le seuil critique, appliqués uniformément au CPU, à la RAM et au disque). Lorsqu'un seuil est franchi, le service `AlertService` génère un document `Alert` et déclenche l'envoi d'un courrier électronique de notification à l'administrateur, aussi bien pour le niveau WARNING que pour le niveau CRITICAL, afin de garantir une réactivité maximale.

Une alerte peut ensuite être consultée puis *acquittée* par l'administrateur depuis l'interface, ce qui trace formellement sa prise en charge sans pour autant la supprimer de l'historique.

4.3.2 Actions à distance via SSH

Le module d'actions à distance permet à un administrateur d'intervenir directement sur un serveur supervisé, sans ouverture manuelle d'une session SSH. Trois opérations

sont proposées : redémarrage d'un service, arrêt d'un service, redémarrage complet du serveur.

Sécurité d'accès. Ces actions sont protégées par une **double vérification** : la présence d'un jeton JWT valide (middleware `verifyToken`, appliqué à l'ensemble des routes d'actions à distance) et l'appartenance de l'utilisateur au rôle administrateur, vérifiée en base de données à chaque requête (middleware `requireAdmin`).

Exécution. Une fois la requête autorisée, le backend établit une connexion SSH vers le serveur cible, à l'aide des identifiants (adresse IP, utilisateur, mot de passe, port) stockés sur le document `Server` correspondant, et exécute la commande `systemctl` adéquate (`restart`, `stop`). Le cas particulier du gestionnaire de processus **PM2** — qui n'est pas un service `systemd` — est traité séparément par l'exécution de la commande `pm2 restart all / pm2 stop all`.

Détection dynamique des services. Plutôt que de limiter les actions à une liste de services codée en dur, l'agent Python détecte, sur chaque serveur, l'ensemble des services `systemd` réellement actifs (commande `systemctl list-units -type=service -state=active`) et transmet cette liste au backend à chaque cycle de collecte. Cette liste, persistée sur le document `Server`, est ensuite affichée côté client : seuls les services réellement présents sur un serveur donné y sont proposés à l'administrateur, quel que soit le serveur consulté.

Vérification post-action. Après exécution, le backend interroge à nouveau le serveur (`systemctl is-active`) afin de confirmer l'état réel du service et de retourner un statut vérifié (*Actif*, *Inactif*, *Échec* ou *Inconnu*) au client, plutôt que de se contenter de supposer le succès de la commande exécutée.

Traçabilité. Chaque action est journalisée dans la collection `AuditLog` (auteur, cible, horodatage, adresse IP, résultat) et donne lieu à l'envoi d'un courrier électronique de confirmation.

4.3.3 Gestion des sauvegardes

Les sauvegardes sont déclenchées automatiquement chaque nuit à minuit par une tâche planifiée (`node-cron`). Une vérification complémentaire, exécutée toutes les heures, permet de détecter et de signaler une sauvegarde qui n'aurait pas pu s'exécuter à l'heure prévue. Le résultat de chaque sauvegarde (succès ou échec) est communiqué à l'administrateur par courrier électronique, et son historique est consultable depuis l'interface.

4.3.4 Application mobile (React Native)

L'application mobile, développée avec **React Native** et le framework **Expo** (Expo Router), donne accès depuis un smartphone à l'essentiel des fonctionnalités de la plateforme, en consommant la *même* API REST que le client web. Elle s'articule autour de cinq écrans :

1. **Connexion** — authentification par email et mot de passe, jeton JWT persisté localement (**AsyncStorage**) et validé au démarrage de l'application ;
2. **Tableau de bord** — liste des serveurs et de leur statut, résumé des alertes actives ;
3. **Détail d'un serveur** — métriques en temps réel et graphique d'historique ;
4. **Alertes** — liste des alertes actives et acquittement ;
5. **Actions à distance** — sélection d'un serveur, liste de ses services réellement détectés, avec actions Arrêter / Redémarrer, et redémarrage du serveur.

L'accès aux écrans protégés est conditionné à la présence d'une session valide, au moyen d'un mécanisme de garde de navigation : tant que l'authentification n'est pas confirmée, seul l'écran de connexion est accessible. L'ensemble des écrans se rafraîchit automatiquement toutes les cinq secondes pour les données nécessitant un suivi en temps réel (tableau de bord, détail d'un serveur), reproduisant ainsi le comportement temps réel du client web.

Conclusion

À l'issue de ce sprint, l'application de monitoring est fonctionnellement complète : supervision, alerting, administration à distance, sauvegardes et accessibilité mobile. Les deux sprints suivants portent sur l'industrialisation du déploiement de cette plateforme selon une démarche DevSecOps.

Sprint 3 – Architecture DevSecOps (Partie 1)

Introduction

Les deux sprints précédents ont porté sur la réalisation fonctionnelle de la plateforme de monitoring. Les deux sprints suivants portent sur son **industrialisation** : la mise en place d'une chaîne d'intégration et de déploiement continu (CI/CD) intégrant des contrôles de qualité et de sécurité, préalable à tout déploiement en production. Ce chapitre couvre la première moitié de ce volet : conteneurisation des composants, chaîne d'intégration continue Jenkins, analyse de qualité de code et scan de vulnérabilités.

5.1 Objectif du sprint

L'objectif de ce sprint est de garantir que chaque évolution du code source est automatiquement construite, analysée et validée avant de pouvoir être déployée, réduisant ainsi le risque de mise en production d'un code de qualité insuffisante ou porteur de vulnérabilités connues.

5.2 Analyse du sprint 3

5.2.1 Sprint Backlog

TABLE 5.1 – Sprint Backlog – Sprint 3

ID	Tâche	Statut
T3.1	Rédaction des Dockerfile (backend, frontend, agent)	Terminé
T3.2	Configuration de l'analyse de qualité SonarQube (<code>sonar-project.properties</code>)	Terminé
T3.3	Mise en place du pipeline Jenkins (7 étapes)	Terminé
T3.4	Intégration du scan de vulnérabilités Trivy sur les images construites	Terminé
T3.5	Publication des images sur un registre Docker Hub	Terminé

5.2.2 Diagramme de cas d'utilisation

Le ?? présente les cas d'utilisation de ce sprint, du point de vue de l'équipe technique.

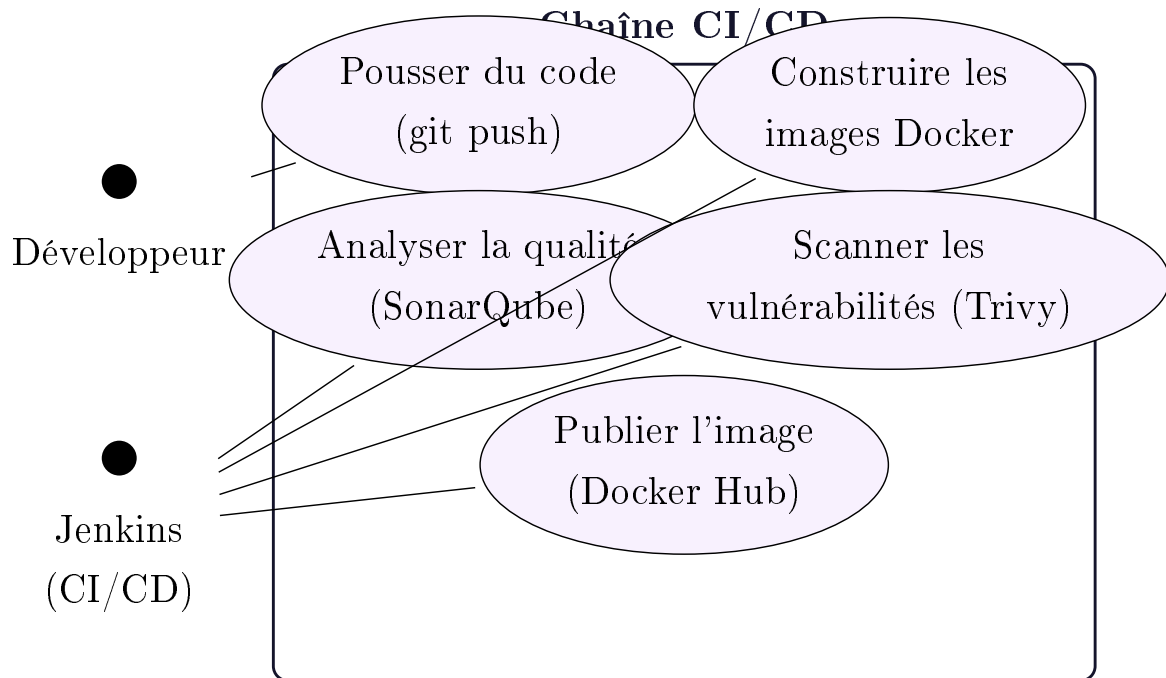


FIGURE 5.1 – Diagramme de cas d'utilisation – Sprint 3

5.3 Réalisation

5.3.1 Conteneurisation avec Docker

Chaque composant applicatif de la plateforme est packagé sous la forme d'une image **Docker** indépendante, construite à partir d'un **Dockerfile** dédié.

Backend. L'image backend repose sur `node:20-slim` et exécute l'application via `pm2-runtime`, ce qui permet de bénéficier de la supervision de processus offerte par PM2 tout en restant compatible avec le modèle d'exécution au premier plan attendu par Docker (??).

```
1 FROM node:20-slim
2 RUN apt-get update && apt-get install -y docker.io
3 RUN npm install -g pm2
4 WORKDIR /app
5 COPY package*.json ./
6 RUN npm install --production
7 COPY . .
8 EXPOSE 5000
9 CMD ["pm2-runtime", "server.js"]
```

Listing 5.1 – Dockerfile du backend

Frontend. L'image frontend s'appuie sur une construction **multi-étapes** : une première étape (`node:20-alpine`) compile l'application React en fichiers statiques, une seconde étape (`nginx:alpine`) ne conserve que ces fichiers statiques et le serveur Nginx chargé de les servir, ce qui permet d'obtenir une image finale nettement plus légère qu'une image contenant l'ensemble de la chaîne de build Node.js.

Agent. L'agent Python est packagé à partir de l'image `python:3.11-slim`, avec installation de ses dépendances (`psutil`, `requests`) via `pip`.

5.3.2 Pipeline CI/CD Jenkins (7 stages)

L'intégration continue est assurée par **Jenkins**, à travers un pipeline déclaratif structuré en sept étapes (*stages*), déclenché automatiquement à chaque poussée de code sur le dépôt Git :

1. **Checkout** — récupération de la dernière version du code source depuis le dépôt Git ;
2. **Installation des dépendances** — installation des paquets Node.js (backend, frontend) et Python (agent) ;
3. **Build** — construction des applications (compilation du frontend React, vérification syntaxique du backend et de l'agent) ;
4. **Analyse de qualité (SonarQube)** — analyse statique du code source, à la recherche de *code smells*, de duplication et de complexité excessive ;
5. **Scan de sécurité (Trivy)** — analyse des images Docker construites à la recherche de vulnérabilités connues ;
6. **Construction et publication des images Docker** — construction des images finales et publication sur le registre Docker Hub ;
7. **Déploiement** — déclenchement du déploiement de la nouvelle version (pris en charge, à ce stade, par le mécanisme GitOps détaillé au chapitre suivant).

Le pipeline est configuré pour interrompre son exécution dès qu’une étape échoue (analyse qualité en dessous du seuil de qualité configuré, vulnérabilité critique détectée), empêchant ainsi la propagation d’une version non conforme vers les étapes suivantes. Le fichier de définition du pipeline (`Jenkinsfile`) est fourni en ??.

5.3.3 Analyse qualité SonarQube

L’analyse de qualité du code source est assurée par **SonarQube**, dont la configuration du projet est définie dans le fichier `sonar-project.properties` à la racine du dépôt (??). Le périmètre d’analyse couvre l’ensemble du code source du projet, à l’exclusion des dépendances installées (`node_modules`) et de l’application mobile, analysée séparément.

```
1 sonar.projectKey=monitoring-app
2 sonar.projectName=DevOps Monitoring App
3 sonar.sources=.
4 sonar.exclusions=**/node_modules/**,**/mobile-app/**,**/.git/**
```

Listing 5.2 – Configuration SonarQube

À l’issue de chaque analyse, SonarQube établit un *Quality Gate* : un ensemble de critères (couverture de test, dette technique, absence de vulnérabilités de sévérité élevée) que le code doit satisfaire pour être considéré comme conforme. Le pipeline Jenkins interroge ce résultat et interrompt la suite du déploiement en cas de non-conformité.

5.3.4 Scan de vulnérabilités Trivy

Une fois les images Docker construites, celles-ci sont analysées par **Trivy**, un scanner de vulnérabilités *open source* spécialisé dans l’analyse des images de conteneurs. Trivy examine le système de fichiers de l’image (dépendances système et applicatives) et le compare à des bases de données de vulnérabilités connues (CVE), afin de détecter la présence de composants obsolètes ou vulnérables avant que l’image ne soit publiée sur le registre. Le pipeline est configuré pour faire échouer la construction en cas de détection d’une vulnérabilité de sévérité **CRITICAL** ou **HIGH**, garantissant qu’aucune image manifestement vulnérable n’atteint le registre Docker Hub.

Conclusion

Ce sprint a permis de mettre en place les fondations de l’intégration continue de la plateforme : conteneurisation des composants, automatisation de la construction, et

intégration de contrôles de qualité et de sécurité systématiques avant toute publication d'image. Le sprint suivant s'appuie sur ces images pour automatiser le déploiement et l'exploitation de la plateforme sur un cluster Kubernetes.

Sprint 4 – Architecture DevSecOps (Partie 2)

Introduction

Ce dernier chapitre technique présente le Sprint 4, consacré au déploiement et à l'exploitation de la plateforme sur un cluster **Kubernetes**, ainsi qu'à la sécurisation et à la supervision de cette infrastructure elle-même.

6.1 Objectif du sprint

L'objectif de ce sprint est de faire passer la plateforme d'un déploiement manuel à un déploiement **orchestré, reproductible et supervisé** : orchestration des conteneurs, déploiement continu piloté par l'état du dépôt Git (GitOps), gestion sécurisée des secrets, contrôle d'accès au cluster, et supervision de l'infrastructure elle-même.

6.2 Analyse du sprint 4

6.2.1 Sprint Backlog

TABLE 6.1 – Sprint Backlog – Sprint 4

ID	Tâche	Statut
T4.1	Installation et configuration du cluster Kubernetes léger (k3s) sur le VPS OVH	Terminé
T4.2	Rédaction des manifests Kubernetes (Deployment, Service)	Terminé
T4.3	Mise en place du déploiement continu GitOps avec Argo CD	Terminé
T4.4	Externalisation et sécurisation des secrets avec HashiCorp Vault	Terminé
T4.5	Configuration du contrôle d'accès RBAC sur le cluster	Terminé
T4.6	Déploiement de la supervision d'infrastructure Prometheus + Grafana	Terminé

6.2.2 Diagramme de cas d'utilisation

Le ?? présente les cas d'utilisation couverts par ce sprint.

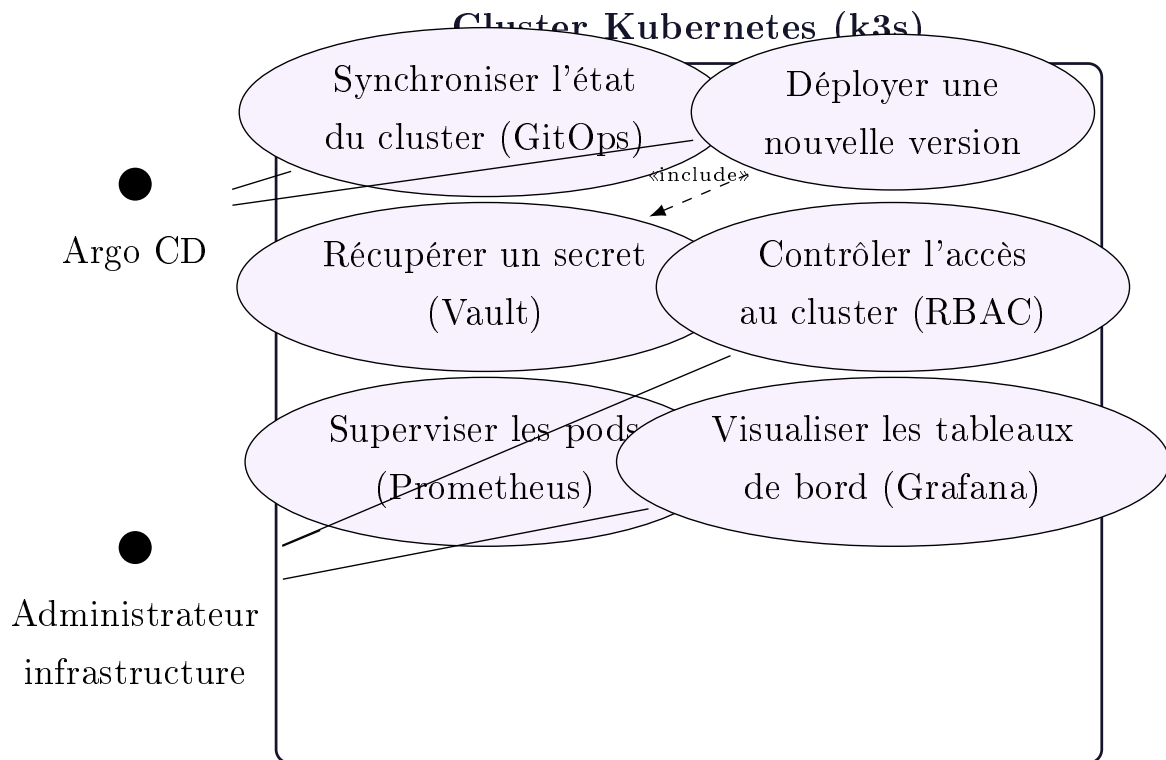


FIGURE 6.1 – Diagramme de cas d'utilisation – Sprint 4

6.3 Réalisation

6.3.1 Déploiement Kubernetes (k3s)

La plateforme est déployée sur un cluster **Kubernetes** allégé (**k3s**), installé directement sur le VPS OVH. Ce choix permet de bénéficier des mécanismes d'orchestration standards de Kubernetes (gestion déclarative des ressources, redémarrage automatique des conteneurs défaillants, mise à l'échelle) tout en conservant une empreinte système compatible avec un unique serveur de taille modeste.

Chaque composant applicatif fait l'objet d'un **Deployment** et d'un **Service** Kubernetes dédiés. Le backend et le frontend sont exposés à l'extérieur du cluster via des services de type **NodePort** (??), tandis que la base de données MongoDB reste accessible uniquement au sein du cluster. Les identifiants de connexion à la base de données (MongoDB Atlas) sont injectés dans le pod backend au moyen d'un objet **Secret** Kubernetes, plutôt que d'être codés en dur dans l'image ou le manifeste de déploiement.

TABLE 6.2 – Exposition des services Kubernetes

Service	Rôle	NodePort
frontend	Application web React (servie par Nginx)	30080
backend	API REST + WebSocket (Node.js / Express)	30300

L'ensemble des manifestes Kubernetes du projet est fourni en ??.

6.3.2 GitOps avec Argo CD

Le déploiement continu est piloté selon une approche **GitOps**, à l'aide d'**Argo CD**. Le principe consiste à faire du dépôt Git contenant les manifestes Kubernetes la *source de vérité unique* de l'état souhaité du cluster : Argo CD surveille en continu ce dépôt et réconcilie automatiquement l'état réel du cluster avec l'état déclaré, sans intervention manuelle de type `kubectl apply`. Cette approche apporte une traçabilité complète des évolutions de l'infrastructure (chaque changement d'état correspond à un commit Git) et facilite le retour arrière en cas de déploiement défectueux.

6.3.3 Gestion des secrets HashiCorp Vault

Afin de renforcer la sécurité de la gestion des informations sensibles (identifiants de base de données, identifiants SSH des serveurs supervisés), la plateforme s'appuie sur **HashiCorp Vault** pour centraliser le stockage et la distribution de ces secrets, plutôt que de les faire transiter uniquement par des objets **Secret** Kubernetes statiques. Vault permet une gestion centralisée du cycle de vie des secrets (rotation, révocation, contrôle d'accès fin) indépendamment du cycle de vie des applications qui les consomment.

6.3.4 Contrôle d'accès RBAC

L'accès au cluster Kubernetes est régi par un mécanisme de contrôle d'accès basé sur les rôles (**RBAC** – *Role-Based Access Control*). Des rôles (**Role** / **ClusterRole**) définissent des ensembles de permissions sur les ressources du cluster (consultation, modification, suppression), qui sont ensuite associés aux comptes de service ou aux utilisateurs via des liaisons (**RoleBinding** / **ClusterRoleBinding**). Cette approche permet d'appliquer le principe du moindre privilège : chaque composant (par exemple, Argo CD) ne dispose que des permissions strictement nécessaires à son fonctionnement.

6.3.5 Monitoring Prometheus + Grafana

Au-delà du monitoring applicatif propre à la plateforme (objet des chapitres 3 et 4), l'infrastructure d'exécution elle-même est supervisée à l'aide de la stack **Prometheus**

+ **Grafana**, standard de l'écosystème Kubernetes : Prometheus collecte périodiquement les métriques exposées par les composants du cluster (état des pods, consommation de ressources), tandis que Grafana permet de construire des tableaux de bord de visualisation de ces métriques. Cette supervision de l'infrastructure est complémentaire à celle assurée par l'agent Python développé au chapitre 3 : elle porte sur la santé du cluster d'exécution, là où l'agent porte sur la santé des serveurs supervisés par la plateforme elle-même. Un exemple de configuration Prometheus est fourni en ??.

Conclusion

Ce sprint conclut le volet DevSecOps du projet : la plateforme de monitoring, développée au cours des Sprints 1 et 2, est désormais déployée de façon orchestrée et reproductible sur un cluster Kubernetes, avec une gestion sécurisée des secrets, un contrôle d'accès formalisé, et une supervision de l'infrastructure elle-même. Le chapitre suivant dresse le bilan général du projet.

Conclusion Générale

Le présent projet de fin d'études a consisté à concevoir, développer et mettre en production une plateforme complète de **monitoring et de gestion à distance** d'infrastructure serveur pour le compte de CLEDISS, structurée selon une démarche agile Scrum en cinq sprints.

Les deux premiers sprints applicatifs ont permis de bâtir une chaîne complète de supervision : un agent Python léger, installé sur chaque serveur supervisé, collecte en continu les métriques système et détecte automatiquement les services actifs ; un backend Node.js centralise ces données, calcule le statut de chaque serveur, génère des alertes lors du dépassement de seuils configurables et notifie l'administrateur par courrier électronique ; un client web React et un client mobile React Native, tous deux consommant la même API REST protégée par authentification JWT, offrent une expérience homogène de supervision et d'administration, y compris depuis un smartphone. Le module d'actions à distance, sécurisé par une double vérification d'authentification et intégralement journalisé, permet d'intervenir directement sur un service ou un serveur sans ouverture manuelle d'une session SSH, tandis que le module de sauvegarde garantit la disponibilité des données par une exécution planifiée et notifiée.

Les deux sprints suivants ont porté sur l'industrialisation du déploiement de cette plateforme selon une démarche **DevSecOps** : conteneurisation de chaque composant, chaîne d'intégration continue Jenkins intégrant systématiquement une analyse de qualité (SonarQube) et un scan de vulnérabilités (Trivy) avant toute publication d'image, puis déploiement orchestré sur un cluster Kubernetes, piloté par une approche GitOps (Argo CD), avec une gestion centralisée des secrets (Vault), un contrôle d'accès formalisé (RBAC) et une supervision de l'infrastructure elle-même (Prometheus, Grafana).

Ce projet a ainsi permis de couvrir l'intégralité du cycle de vie d'une plateforme logicielle moderne : de la conception fonctionnelle à l'exploitation en production, en passant par la sécurisation de chacune de ses étapes. Sur le plan personnel, il a été l'occasion d'approfondir des compétences en développement full-stack (Node.js, React, React Native), en administration système (SSH, systemd), ainsi qu'en intégration et déploiement continus, autant de compétences directement mobilisables dans un contexte professionnel.

Perspectives

Plusieurs axes d'évolution peuvent être envisagés pour prolonger ce travail :

- **alerting avancé** — introduction de règles d'alerte personnalisables par serveur et de mécanismes de corrélation entre plusieurs métriques ;
- **historisation et analyse de tendances** — exploitation de l'historique des métriques à des fins d'analyse prédictive (anticipation d'une saturation de ressources) ;
- **extension multi-environnements** — généralisation du support à d'autres fournisseurs cloud, au-delà d'OVH ;
- **gestion fine des rôles** — introduction de rôles utilisateurs intermédiaires entre lecture seule et administration complète ;
- **tests automatisés** — renforcement de la couverture de tests automatisés (unitaires et d'intégration) intégrés à la chaîne CI/CD, afin de fiabiliser davantage encore les déploiements continus.

Bibliographie

- [1] Node.js Foundation, *Node.js Documentation*. <https://nodejs.org/docs>
- [2] OpenJS Foundation, *Express – Node.js web application framework*. <https://expressjs.com>
- [3] Socket.IO, *Socket.IO Documentation*. <https://socket.io/docs>
- [4] MongoDB Inc., *MongoDB Documentation*. <https://www.mongodb.com/docs>
- [5] Mongoose, *Mongoose ODM Documentation*. <https://mongoosejs.com/docs>
- [6] Meta Open Source, *React Documentation*. <https://react.dev>
- [7] Meta Open Source, *React Native Documentation*. <https://reactnative.dev/docs>
- [8] Expo, *Expo Documentation*. <https://docs.expo.dev>
- [9] Python Software Foundation, *The Python Standard Library*. <https://docs.python.org>
- [10] G. Rodola, *psutil Documentation*. <https://psutil.readthedocs.io>
- [11] Internet Engineering Task Force, *RFC 7519 – JSON Web Token (JWT)*. <https://datatracker.ietf.org/doc/html/rfc7519>
- [12] Docker Inc., *Docker Documentation*. <https://docs.docker.com>
- [13] Cloud Native Computing Foundation, *Kubernetes Documentation*. <https://kubernetes.io/docs>
- [14] Rancher Labs, *K3s – Lightweight Kubernetes Documentation*. <https://docs.k3s.io>
- [15] Jenkins Project, *Jenkins User Documentation*. <https://www.jenkins.io/doc>
- [16] SonarSource, *SonarQube Documentation*. <https://docs.sonarsource.com>
- [17] Aqua Security, *Trivy Documentation*. <https://aquasecurity.github.io/trivy>
- [18] Argo Project, *Argo CD – Declarative GitOps CD for Kubernetes*. <https://argo-cd.readthedocs.io>
- [19] HashiCorp, *Vault Documentation*. <https://developer.hashicorp.com/vault/docs>
- [20] Cloud Native Computing Foundation, *Using RBAC Authorization – Kubernetes Documentation*. <https://kubernetes.io/docs/reference/access-authn-authz/rbac>

- [21] Prometheus Authors, *Prometheus Documentation*. <https://prometheus.io/docs>
- [22] Grafana Labs, *Grafana Documentation*. <https://grafana.com/docs>
- [23] K. Schwaber, J. Sutherland, *The Scrum Guide*. <https://scrumguides.org>

Annexe A – Jenkinsfile

Le script ci-dessous présente la structure du pipeline déclaratif Jenkins, correspondant aux sept étapes décrites en ?? . *Ce fichier est un modèle représentatif de la structure du pipeline ; il doit être remplacé par le contenu exact du Jenkinsfile du projet avant soutenance.*

```
1 pipeline {
2   agent any
3
4   environment {
5     DOCKERHUB_CREDENTIALS = credentials('dockerhub-credentials')
6     SONAR_TOKEN            = credentials('sonarqube-token')
7   }
8
9   stages {
10
11     stage('1. Checkout') {
12       steps {
13         checkout scm
14       }
15     }
16
17     stage('2. Installation des dependances') {
18       steps {
19         sh 'cd backend && npm install'
20         sh 'cd frontend && npm install'
21         sh 'cd agent && pip install -r requirements.txt'
22       }
23     }
24
25     stage('3. Build') {
26       steps {
27         sh 'cd frontend && npm run build'
28       }
29     }
30
31     stage('4. Analyse qualite (SonarQube)') {
32       steps {
33         withSonarQubeEnv('sonarqube-server') {
34           sh 'sonar-scanner'
35         }
36       }
37     }
38   }
39 }
```

```
38
39 stage('5. Scan securite (Trivy)') {
40     steps {
41         sh 'docker build -t mariemchaabani/monitoring-backend:${
42             BUILD_NUMBER} ./backend'
43         sh 'trivy image --severity CRITICAL,HIGH --exit-code 1
44             mariemchaabani/monitoring-backend:${BUILD_NUMBER}'
45     }
46 }
47
48 stage('6. Build et push des images Docker') {
49     steps {
50         sh 'docker build -t mariemchaabani/monitoring-frontend:${
51             BUILD_NUMBER} ./frontend'
52         sh 'docker login -u $DOCKERHUB_CREDENTIALS_USR -p
53             $DOCKERHUB_CREDENTIALS_PSW'
54         sh 'docker push mariemchaabani/monitoring-backend:${
55             BUILD_NUMBER}'
56         sh 'docker push mariemchaabani/monitoring-frontend:${
57             BUILD_NUMBER}'
58     }
59 }
60
61 stage('7. Deploiement (GitOps)') {
62     steps {
63         sh 'git commit -am "chore: bump image tag to ${BUILD_NUMBER}"
64             && git push'
65         // La synchronisation effective du cluster est ensuite prise
66         // en
67         // charge par Argo CD (voir Chapitre 6).
68     }
69 }
70
71 post {
72     failure {
73         echo 'Pipeline en echec -- deploiement annule.'
```

Listing 1 – Structure du pipeline Jenkins (modèle)

Annexe B – Manifests Kubernetes

Les manifestes ci-dessous correspondent aux ressources Kubernetes réellement déployées sur le cluster k3s du VPS OVH, référencées au *??*. *Par mesure de sécurité, les valeurs du manifeste **Secret** sont remplacées par des espaces réservés : les valeurs réelles ne doivent jamais figurer dans ce rapport ni dans le dépôt Git du projet.*

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: backend
5   namespace: monitoring
6 spec:
7   replicas: 1
8   selector:
9     matchLabels:
10      app: backend
11   template:
12     metadata:
13       labels:
14         app: backend
15     spec:
16       containers:
17         - name: backend
18           image: mariemchaabani/monitoring-backend:<tag>
19           ports:
20             - containerPort: 3000
21           envFrom:
22             - secretRef:
23                 name: monitoring-secret
24           volumeMounts:
25             - name: backups
26               mountPath: /app/backups
27       volumes:
28         - name: backups
29           hostPath:
30             path: /home/debian/DevOps-Monitoring-App/backups
31 ---
32 apiVersion: v1
33 kind: Service
34 metadata:
35   name: backend
36   namespace: monitoring
37 spec:
```

```
38 selector:
39   app: backend
40 ports:
41   - port: 3000
42     targetPort: 3000
43 nodePort: 30300
44 type: NodePort
```

Listing 2 – Deployment et Service – Backend

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: frontend
5    namespace: monitoring
6  spec:
7    replicas: 1
8    selector:
9      matchLabels:
10       app: frontend
11    template:
12      metadata:
13        labels:
14          app: frontend
15      spec:
16        containers:
17          - name: frontend
18            image: mariemchaabani/monitoring-frontend:<tag>
19            ports:
20              - containerPort: 80
21 ---
22 apiVersion: v1
23 kind: Service
24 metadata:
25   name: frontend
26   namespace: monitoring
27 spec:
28   selector:
29     app: frontend
30   ports:
31     - port: 80
32       targetPort: 80
33   nodePort: 30080
34   type: NodePort
```

Listing 3 – Deployment et Service – Frontend

```
1  apiVersion: v1
```

```
2 kind: Secret
3 metadata:
4   name: monitoring-secret
5   namespace: monitoring
6 type: Opaque
7 stringData:
8   MONGODB_URI: "<mongodb-atlas-connection-string>"
9   EMAIL_USER: "<email-expediteur>"
10  EMAIL_PASS: "<mot-de-passe-application>"
11  PORT: "3000"
12  JWT_SECRET: "<cle-secrete-jwt>"
13  ADMIN_EMAIL: "<email-admin>"
14  ADMIN_PASSWORD: "<mot-de-passe-admin>"
```

Listing 4 – Secret – configuration sensible (valeurs redigees)

Annexe C – Configuration Prometheus

L'extrait ci-dessous illustre une configuration standard de **Prometheus** pour la découverte et la supervision des pods d'un cluster Kubernetes, telle qu'évoquée au ??.

À remplacer par la configuration exacte déployée sur le cluster avant soutenance.

```
1 global:
2   scrape_interval: 15s
3   evaluation_interval: 15s
4
5 scrape_configs:
6   - job_name: 'kubernetes-pods'
7     kubernetes_sd_configs:
8       - role: pod
9     relabel_configs:
10      - source_labels: [
11          __meta_kubernetes_pod_annotation_prometheus_io_scrape]
12        action: keep
13        regex: true
14      - source_labels: [__meta_kubernetes_namespace]
15        target_label: namespace
16      - source_labels: [__meta_kubernetes_pod_name]
17        target_label: pod
18
19   - job_name: 'monitoring-backend'
20     static_configs:
21       - targets: ['backend.monitoring.svc.cluster.local:3000']
```

Listing 5 – Extrait de configuration prometheus.yml

Les métriques ainsi collectées par Prometheus sont ensuite exploitées par **Grafana** pour la construction de tableaux de bord de supervision de l'infrastructure (état des pods, consommation CPU/mémoire par conteneur, disponibilité des services).